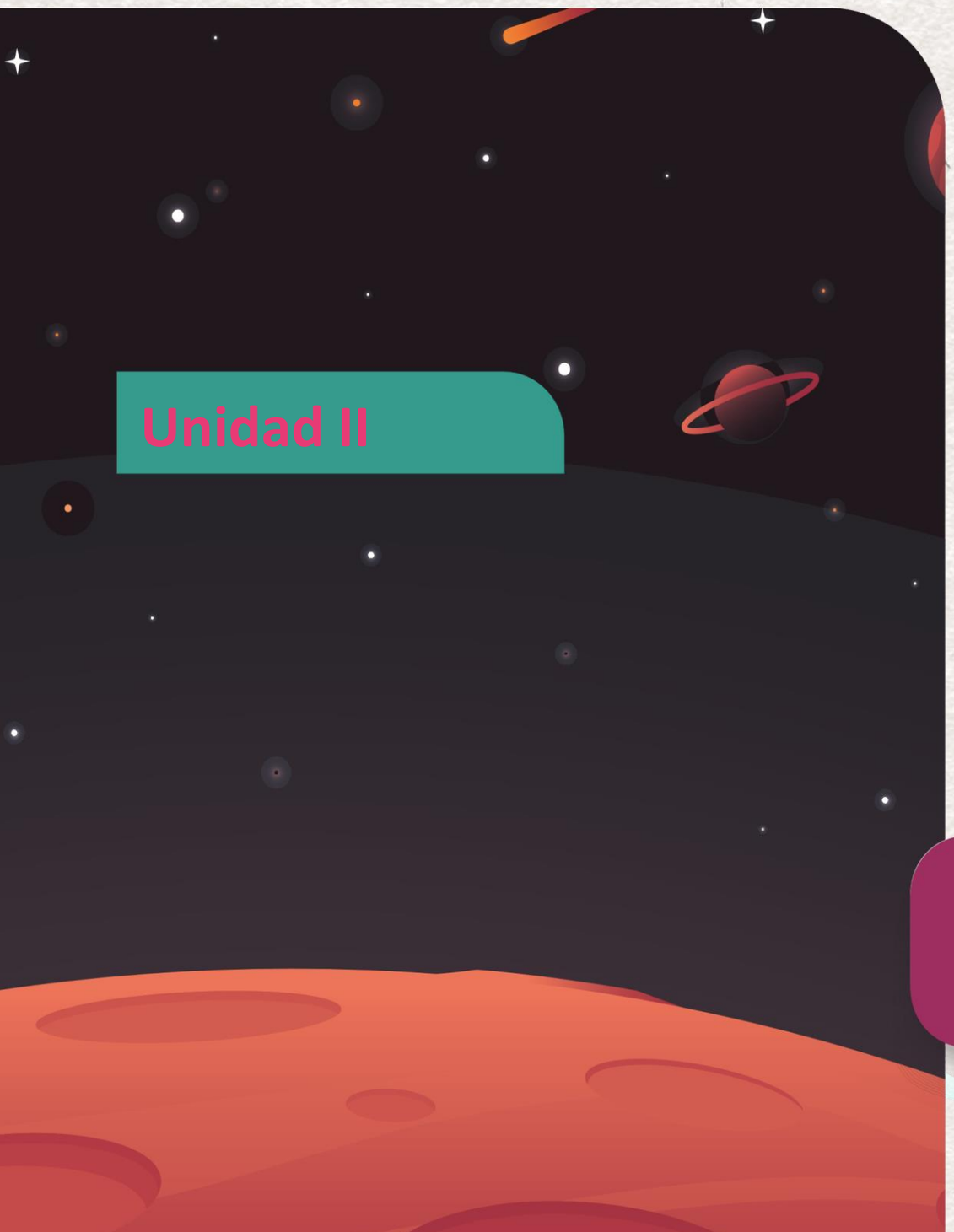
CESDE

Santiago Yosa

**Ingeniero Sistemas de Información**

CESDE | comfama



Unidad II

**CLASES ABSTRACTAS**

¿Qué es una clase abstracta?

Métodos abstractos

Restricciones de las clases abstractas

**SISTEMA CONTROL DE VERSIONES**

-Utilizar git/GitHub para versionar código realizado

# Fundamentos de POO

## Reto rápido La "Falsa" Ejecución

Si ejecuto esto, ¿qué se imprime en la consola y cuál es el problema lógico (de negocio) aquí?

```
public class Animal {
    public void hacerSonido() {
        System.out.println("Sonido genérico de animal...");
    }
}
// En el Main:
Animal a = new Animal();
a.hacerSonido();
```

## Situación

Imagina que estás desarrollando un programa de diseño gráfico. Necesitas manejar diferentes figuras geométricas: círculos, rectángulos, triángulos, etc. **Tu primer impulso es crear una clase para cada una.**
Pero pronto te das cuenta de que todas comparten características y acciones comunes.
Por ejemplo, **toda figura tiene un color** y **toda figura debería poder calcular su área**.
Entonces, piensas en crear una clase padre Figura para agrupar lo común. Pero te enfrentas a una pregunta fundamental: ¿cómo implementas el método calcularArea() en la clase Figura? No existe una fórmula para el área de una "figura genérica".

La fórmula depende enteramente de si es un círculo (πr2), un rectángulo ( base×altura), u otra forma.

Además, no tiene sentido crear un objeto de tipo "Figura" en tu programa. ¿Qué dibujarías en la pantalla?

El concepto de "Figura" es una idea, una abstracción. Solo las figuras **concretas** como un círculo o un rectángulo pueden existir y ser dibujadas.

Aquí es donde necesitas una plantilla que diga:

"Cualquier cosa que sea una figura en este programa **debe tener un color** y **está obligada a tener un método para calcular su área**, pero yo, como plantilla, no puedo decirte cómo hacerlo".

## ¿Qué es una Clase Abstracta?

- Es una clase que sirve como **plantilla** o **base** para otras clases.
- Se declara usando la palabra clave abstract.
- Su propósito es ser heredada (extends) por otras clases, conocidas como **subclases concretas**.
- **Restricción #1:** No se pueden crear objetos (new) de una clase abstracta. El compilador lo bloqueará. Es decir, no puedes crear un objeto directamente de ella (new Figura() resultará en un error de compilación).
- **Permiso #1:** Puede tener atributos normales, métodos normales (con cuerpo) y constructores.
- **Permiso #2:** Sus constructores sí se ejecutan, pero solo a través de la llamada super() desde el constructor de una clase hija.

## ¿Qué es un Método Abstracto?

- Es un método que se declara dentro de una clase abstracta pero no tiene implementación (no tiene cuerpo, ni llaves {}). Termina con un punto y coma.
- Se declara también con la palabra clave abstract.
- Actúa como un contrato o una obligación. Cualquier subclase concreta (no abstracta) que herede de la clase abstracta está obligada a proporcionar una implementación para todos los métodos abstractos heredados.

## La Combinación Perfecta: Métodos Abstractos y Concretos

- La verdadera fortaleza de una clase abstracta es que puede contener **ambos tipos de métodos**:
  - **Métodos Concretos:** Métodos con implementación completa. Este es el código que quieres **compartir** con todas las subclases.
  - **Métodos Abstractos:** Métodos sin implementación. Estas son las funcionalidades que **obligas** a cada subclase a implementar a su propia manera.

## Restricciones Importantes

- Si una clase contiene **uno o más métodos abstractos**, la clase entera **debe** ser declarada como abstract.
- Un método abstracto **no puede ser final**. final significa "no se puede sobrescribir", y abstract significa "debe ser sobrescrito". Son conceptos opuestos.
- Un método abstracto **no puede ser private**. Si fuera privado, ninguna subclase podría verlo y, por lo tanto, no podría implementarlo, rompiendo el contrato.

**Anatomía:**

```
// La palabra abstract va antes de class
public abstract class EntidadBancaria {
    // 1. Atributos normales permitidos
    protected String titular;

    // 2. Constructor permitido (solo invocable vía super())
    public EntidadBancaria(String titular) {
        this.titular = titular;
    }

    // 3. Método normal con cuerpo permitido
    public void imprimirComprobante() {
        System.out.println("Comprobante a nombre de: " + titular);
    }
}
```

# Ejemplos

```java
// Archivo: Figura.java (El Plano Maestro)
public abstract class Figura {
   protected String color;

   public Figura(String color) {
      this.color = color;
   }

   // Método Concreto: Compartido por todas las figuras.
   public void mostrarColor() {
      System.out.println("El color de la figura es: " + this.color);
   }

   // Método Abstracto: Obliga a las subclases a implementarlo.
   public abstract double calcularArea();
}

// Archivo: Circulo.java (Una implementación concreta del plano)
public class Circulo extends Figura {
   private double radio;

   public Circulo(String color, double radio) {
      super(color);
      this.radio = radio;
   }

   // Implementación obligatoria del método abstracto
   @Override
   public double calcularArea() {
      return Math.PI * radio * radio;
   }
}

// Archivo: Main.java
public class Main {
   public static void main(String[] args) {
      // Figura miFigura = new Figura("azul"); // ERROR! No se puede instanciar.

      Circulo miCirculo = new Circulo("Rojo", 5.0);
      miCirculo.mostrarColor(); // Llama al método concreto heredado
      double area = miCirculo.calcularArea(); // Llama al método implementado
      System.out.println("El área del círculo es: " + area);
   }
}
```

## Preguntas

**¿Por qué no usar simplemente una clase normal y dejar los métodos vacíos para que las hijas los sobrescriban?**

Porque eso es peligroso y propenso a errores. Si una superclase normal tiene un método calcularArea() {} (con cuerpo vacío), y el programador de la subclase olvida sobrescribirlo, el programa compilará y se ejecutará sin problemas. Pero cuando se llame a ese método, devolverá cero o nada, causando un bug silencioso. Una clase abstracta con un método abstract convierte ese error lógico en un **error de compilación**, forzando al programador a implementar el método antes de poder ejecutar el programa. Es un mecanismo de seguridad.

**¿Qué te obliga a hacer el compilador si creas una nueva clase Triangulo que hereda de Figura, y por qué es esto útil?**

El compilador te obliga a implementar el método public double calcularArea(). Esto es útil porque garantiza que no puedas crear una clase de figura "incompleta". Te asegura que cada figura en tu sistema tendrá la capacidad de calcular su área, manteniendo la consistencia y previniendo errores en tiempo de ejecución.

## Práctica

Construir un sistema base para diferentes tipos de personajes en un videojuego.

### Paso 1: Configura tu Entorno y Git

1. Crea una nueva carpeta llamada JuegoRPG.
2. Ábrela en tu editor y/o en la terminal, ejecuta git init.
3. En el archivo .gitignore y añade *.class y las carpetas de tu IDE (ej. /.idea/).

### Paso 2: El Plano Maestro del Personaje

1. Crea el proyecto en java.
2. Crea un archivo Personaje.java.
3. Decláralo como public abstract class Personaje.
4. Añadele atributos protected: String nombre y int puntosDeVida.
5. Crea un constructor para inicializar estos atributos.
6. Añade un **método concreto**: public void recibirDanio(int danio) que reste el daño a los puntos de vida e imprima un mensaje.
7. Añade un **método abstracto**: public abstract void usarHabilidadEspecial();

**Paso 3: Versiona tu Progreso**

**Paso 4: Crea los Personajes Concretos**

1. Crea Guerrero.java que extienda Personaje.
2. Crea su constructor y asegúrate de llamar a super().
3. Implementa el método usarHabilidadEspecial() con un System.out.println("El guerrero usa 'Ataque Giratorio'!").
4. Haz lo mismo para una clase Mago.java, pero su habilidad especial será "El mago lanza una 'Bola de Fuego'!".

**Paso 5: La Prueba en Main.java**

1. En la App.java.
2. Crea una lista de tipo Personaje: ArrayList<Personaje> personajes.
3. Añade un nuevo Guerrero y un nuevo Mago a la lista.
4. Recorre la lista con un bucle for-each y para cada personaje, llama tanto a recibirDanio(10) como a usarHabilidadEspecial(). Observa cómo se ejecuta el método concreto compartido y los métodos especializados de cada uno.

**Paso 6: Sube tu Código a GitHub**

## Troubleshooting

### A. Errores de ejecución (Cómo leer la consola):

**Error:** Cannot instantiate the type X

- *Visión en Consola:* Al compilar la línea Figura f = new Figura();
- *Lectura:* Figura tiene la palabra abstract. Intenta hacer el new de una clase hija (ej. new Cuadrado()).

**Error:** The type X must implement the inherited abstract method Y

- *Visión en Consola:* Al crear class Cuadrado extends Figura { }
- *Lectura:* La clase Padre Figura tiene un método abstracto que tú no has sobrescrito en Cuadrado. El compilador te exige que cumplas el contrato.

## Dinámica de "Mob Debugging"

La clase Bicicleta marca error en la línea 1, diciendo que "no implementa el método heredado". Encuentren por qué, si claramente ven que el método moverse está escrito.

```
abstract class Vehiculo {
   public abstract void moverse(int velocidad);
}

class Bicicleta extends Vehiculo {

   // El desarrollador intentó cumplir el contrato:
   public void moverse(double velocidad) {
      System.out.println("Pedaleando a " + velocidad + " km/h");
   }
}
```

El método en el padre pide un int. El hijo implementó uno que recibe double. Para Java, esos son dos métodos totalmente diferentes (Sobrecarga, no Sobrescritura). Como el hijo no sobrescribió exactamente moverse(int), no ha cumplido el contrato. Exígeles usar siempre @Override para que el IDE les avise de este error de firma.

## Práctica Integradora

### A. Instrucciones (Ticket de Jira):

**TICKET-404: Sistema de Exportación Documental Descripción:** Necesitamos un motor genérico para exportar reportes. Todo reporte tiene un nombreArchivo y un método prepararCabeceras(). Sin embargo, la forma en que se genera el archivo en disco (exportar()) depende de si es PDF o Excel.

**Criterios de Aceptación:**

1. Nadie debe poder instanciar un "Reporte Genérico".
2. Forzar arquitectónicamente a que cualquier nuevo formato de reporte (PDF, Excel) implemente el método exportar().

### B. Código Base (Pair Programming - Cacería de Bugs):

Un estudiante digita, el resto de la clase dicta. El siguiente código tiene 3 errores conceptuales graves.

Encuéntrenlos y repárenlos.

```java
// BUG 1: Falta algo en la definición de la clase
public class Reporte {
    protected String nombreArchivo;

    public Reporte(String nombre) { this.nombreArchivo = nombre; }

    public void prepararCabeceras() {
        System.out.println("Generando cabeceras corporativas...");
    }

    // BUG 2: Un método abstracto no puede tener llaves { }
    public abstract void exportar() {
        System.out.println("Exportando...");
    }
}

// BUG 3: El hijo intenta heredar de Reporte pero le falta cumplir algo crítico
public class ReportePDF extends Reporte {
    public ReportePDF(String nombre) {
        super(nombre);
    }
    // Falta el @Override y el método exportar()
}
```

# Ejercicio Independiente

**A. Situación:** Vas a diseñar el motor de nómina del restaurante "El Gourmet Polimórfico".

**B. Instrucciones:**

1. Crea una clase abstracta EmpleadoBase con los atributos nombre y salarioFijo, y el constructor respectivo.
2. Crea el método concreto mostrarInfo() que imprima el nombre del empleado.
3. Crea el método abstracto public abstract double calcularSalarioTotal();.
4. Crea la clase concreta Mesero (hereda de EmpleadoBase). Añade el atributo propinasSemanales. Su salario total es salarioFijo + propinasSemanales.
5. Crea la clase concreta Repartidor. Añade los atributos entregasRealizadas y pagoPorEntrega. Su salario total es salarioFijo + (entregasRealizadas * pagoPorEntrega).
6. En tu Main, crea un ArrayList<EmpleadoBase>, añade un mesero y un repartidor, e itera la lista calculando y mostrando el salario de todos. Versiona tu código en GitHub.

**C. Criterios de Logro (Evaluación Estricta):**

1. **Validación Abstracta:** El código debe fallar en compilación si se intenta hacer new EmpleadoBase(...).
2. **Cumplimiento de Contrato:** Las clases Mesero y Repartidor deben usar estrictamente @Override sobre calcularSalarioTotal().
3. **Polimorfismo:** El Main no debe contener condicionales (if/else) para saber cómo calcular el salario de cada empleado en la lista.

## ¿Debería saber algo más?

**Advertencia Arquitectónica:**
Muchos Juniors abusan de las clases abstractas. Recuerda la regla del "ES UN" (IS-A). Si los objetos comparten comportamiento pero NO son fundamentalmente de la misma familia, usar una Clase Abstracta es un error de diseño que acopla el código.

**Entrada al siguiente tema:**
Las clases abstractas son geniales, pero en Java **solo puedes heredar de una clase padre** (Herencia Simple). ¿Qué pasa si un Mesero necesita heredar de EmpleadoBase pero también necesita comportarse como un Cajero? Java no nos deja hacer extends EmpleadoBase, Cajero. Para resolver este bloqueo, en la próxima sesión descubriremos el nivel más alto de abstracción en Java: **Las Interfaces (interface)**, que nos permiten la "herencia múltiple" de contratos.

CESDE | comfama